

SI3003 - Inteligencia Artificial

CLASE 5 – PROCESOS DE DECISIÓN DE MARKOV: BELLMAN, VALUE
ITERATION Y POLICY ITERATION

Hoy

- De optimización a decisión secuencial bajo incertidumbre
- Formalización: el MDP como tupla
- La ecuación de Bellman (derivación completa)
- Value Iteration
 - Ejemplo trabajado a mano
 - Live-coding: Gridworld 4×3 estocástico
- Policy Iteration
 - Comparación VI vs. PI
- Cierre: ¿y si no conocemos el modelo?



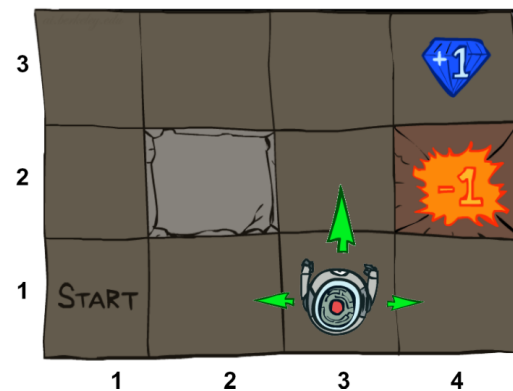
De "encontrar el mejor estado" a "decidir bajo incertidumbre"

En Clase de optimización, un problema de optimización terminaba en **un** estado final elegido de una vez. Hoy el agente no elige un estado — elige **acciones**, una tras otra, y el mundo responde de forma **incierto**: la misma acción puede llevar a resultados distintos. La pregunta ya no es "¿cuál es el mejor estado?" sino "¿cuál es la mejor **acción** dado dónde estoy ahora, sabiendo que el resultado no está garantizado?"

El mundo: Gridworld

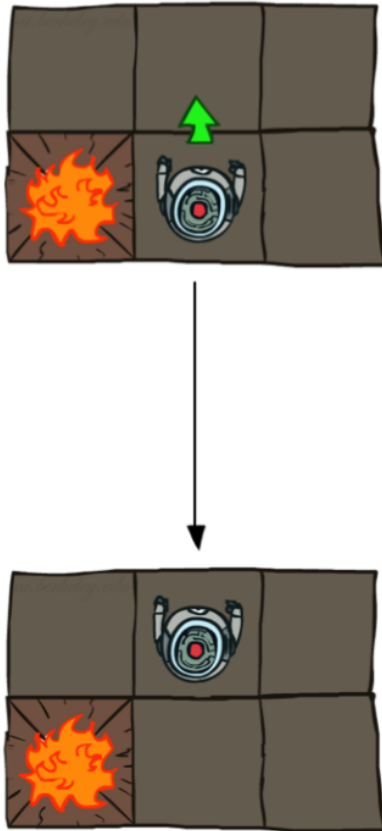
Imaginemos un agente que vive en un ambiente de grilla 3×4 .

- **Movimientos ruidosos:** las acciones no siempre salen como se planean.
 - Cada acción logra el efecto pretendido con probabilidad 0.8.
 - El resto del tiempo, con probabilidad 0.2, la acción mueve al agente en ángulo recto respecto a la dirección pretendida.
 - Si hay una pared en la dirección a la que el agente habría sido llevado, el agente simplemente se queda donde está.
- El agente recibe una recompensa en cada paso de tiempo.
 - Una pequeña recompensa de "vivir" en cada paso (puede ser negativa).
 - Recompensas grandes llegan solo al final (buenas o malas).

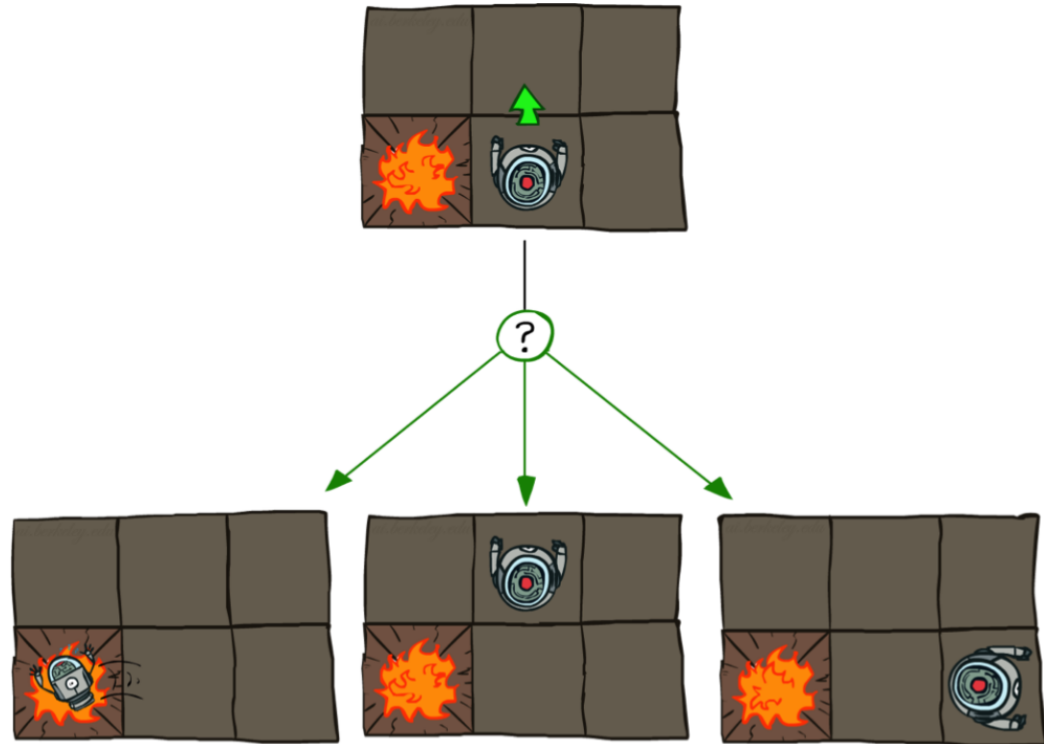


Meta: maximizar la suma de recompensas — no llegar a ningún lugar en particular, solo acumular la mayor recompensa posible en el camino.

Acción determinista



Acción estocástica



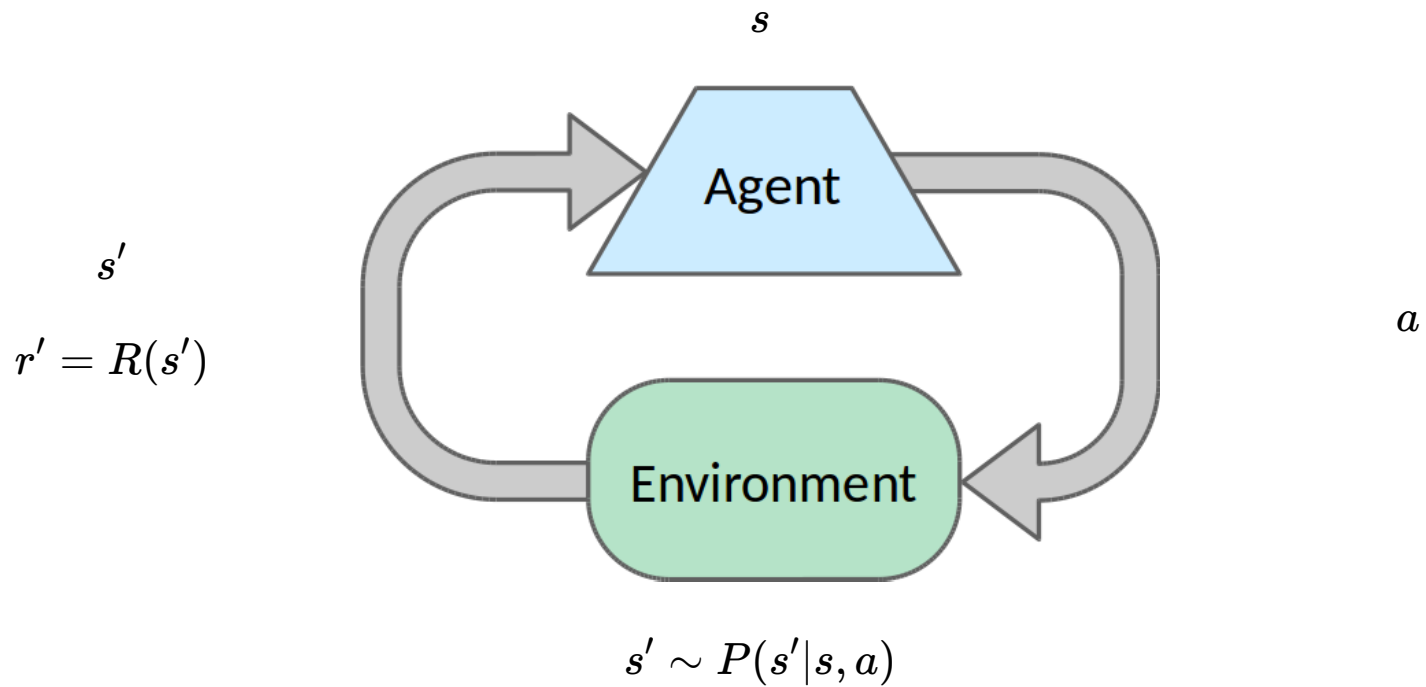
Esta es la diferencia que cambia todo respecto a la búsqueda de Clase 2: ahí, "ir arriba" significaba ir arriba, punto. Aquí, "ir arriba" significa "arriba con 80% de probabilidad, y a los lados con 10% cada uno" — el agente nunca tiene control total sobre a dónde termina.

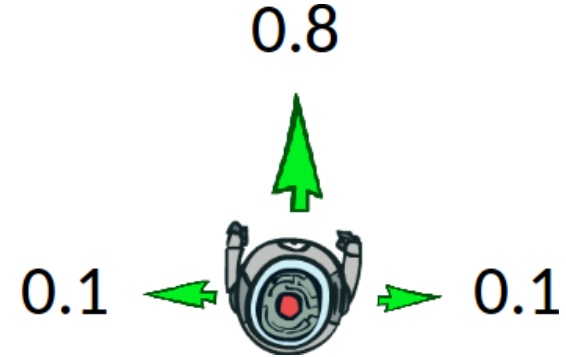
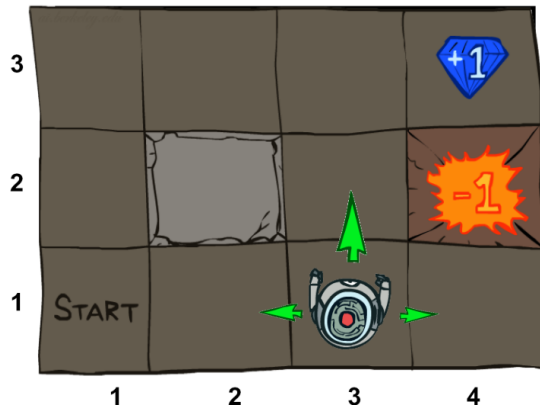
El Proceso de Decisión de Markov (MDP)

Un **MDP** es una tupla (S, A, P, R, γ) :

- S : conjunto de **estados**.
- A : conjunto de **acciones** disponibles.
- $P(s' \mid s, a)$: **modelo de transición**, estacionario — probabilidad de terminar en s' si tomamos la acción a en el estado s .
- $R(s)$: **función de recompensa** del estado. (En su forma más general la recompensa puede depender de la transición completa, $R(s, a, s')$; nosotros usamos la versión simplificada $R(s)$, donde solo depende del estado de llegada.)
- $\gamma \in [0, 1]$: **factor de descuento**.

La **propiedad de Markov**: $P(s' \mid s, a)$ depende solo del estado actual s y la acción a — no de cómo llegamos a s . Todo el historial relevante ya está resumido en s .





Ejemplo

- $\mathcal{S}$: ubicación (i, j) en el grid.
- $\mathcal{A}$: [Arriba, Abajo, Derecha, Izquierda].
- Modelo de transición: $P(s' \mid s, a)$
- Recompensa:

$$R(s) = \begin{cases} -0.4 & \text{para estados no terminales} \\ \pm 1 & \text{para estados terminales} \end{cases}$$

¿Qué tiene de markoviano un MDP?

Dado el estado presente, el futuro y el pasado son independientes:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} \mid s_t, a_t)$$

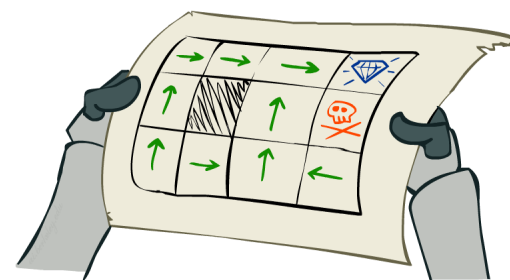
Esto es similar a los problemas de búsqueda (Clase 2), donde la función sucesora solo podía depender del estado actual.



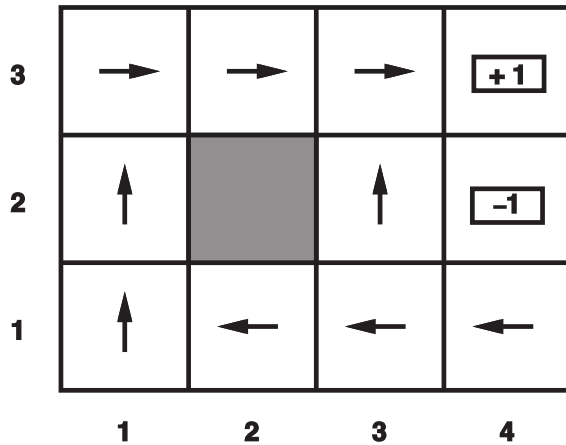
Andrey Markov

Políticas

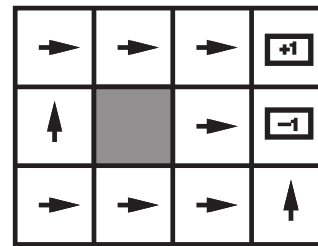
- En problemas de búsqueda determinista de un solo agente (Clase 2), nuestra meta era encontrar un plan óptimo, o *secuencia* de acciones, desde el inicio hasta la meta.
- Para MDPs, queremos encontrar una **política** óptima $\pi^* : \mathcal{S} \rightarrow \mathcal{A}$.
 - Una política π mapea estados a acciones.
 - Una política óptima es la que maximiza la utilidad esperada, es decir, la suma esperada de recompensas.
 - Una política explícita define un agente reflejo.
- Expectiminimax no calculaba políticas completas, solo alguna acción para un único estado.



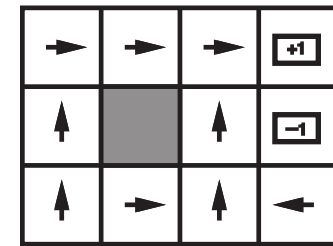
Política óptima cuando $R(s) = -0.3$ para todos los estados no terminales s .



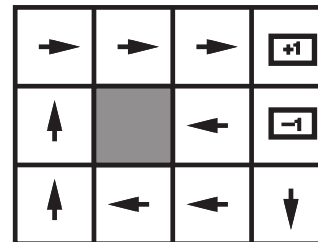
(a)



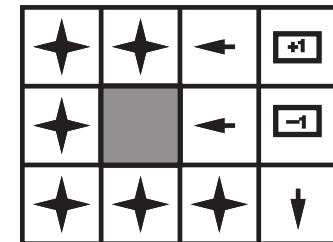
$$R(s) < -1.6284$$



$$-0.4278 < R(s) < -0.0850$$



$$-0.0221 < R(s) < 0$$



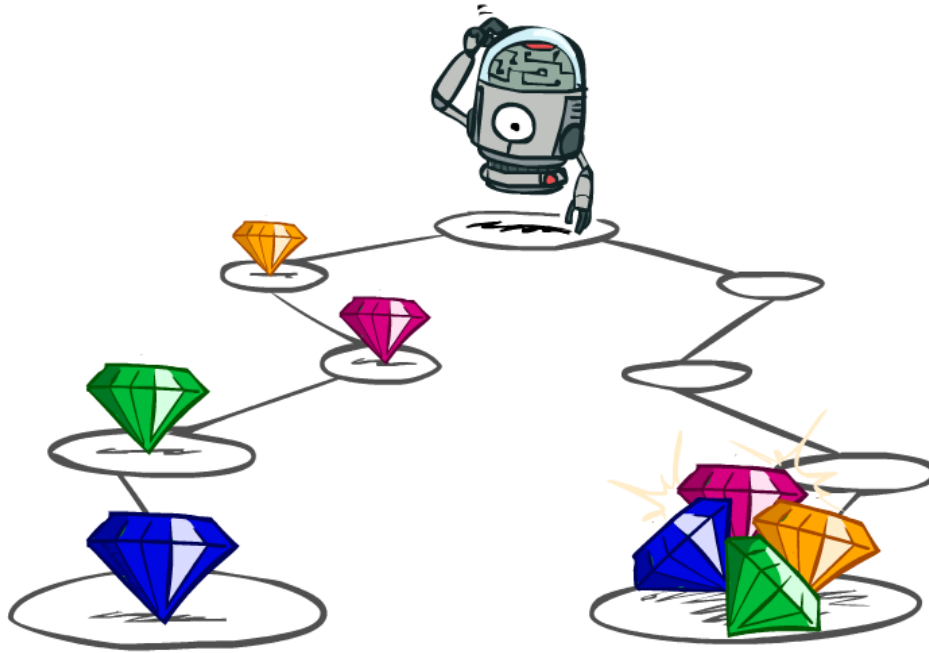
$$R(s) > 0$$

(b)

(a) Política óptima cuando $R(s) = -0.04$ para todos los estados no terminales s . (b) Políticas óptimas para cuatro rangos distintos de $R(s)$.

Dependiendo de $R(s)$, el **balance entre riesgo y recompensa** cambia de arriesgado a muy conservador.

Utilidades a través del tiempo

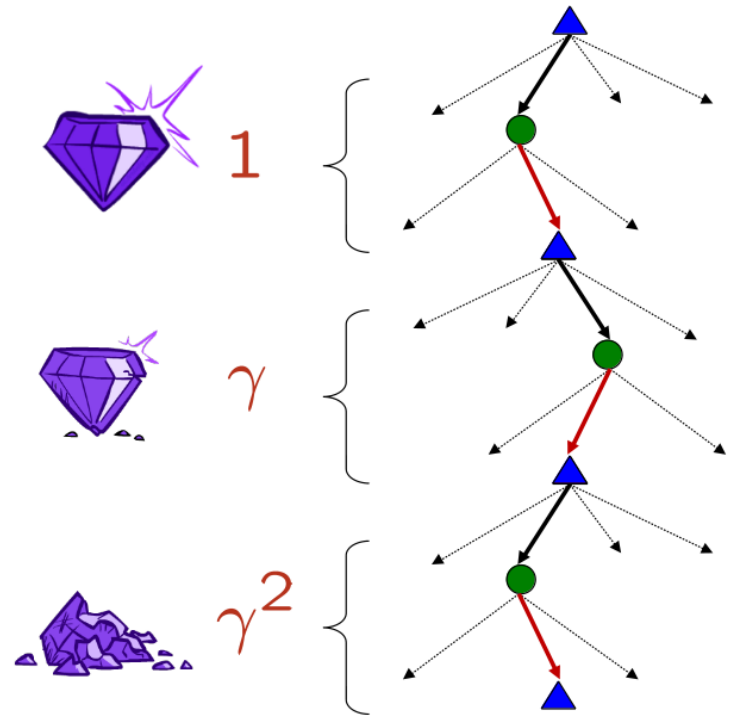


¿Qué preferencias debería tener un agente sobre secuencias de estados o recompensas?

- ¿Más o menos? $[2, 3, 4]$ o $[1, 2, 2]$?
- ¿Ahora o después? $[1, 0, 0]$ o $[0, 0, 1]$?

Descuento

- Cada vez que transicionamos al siguiente estado, multiplicamos el descuento una vez más.
- ¿Por qué descontar?
 - Las recompensas más próximas probablemente tienen mayor utilidad que las recompensas lejanas.
 - Ayuda a que nuestros algoritmos converjan.



Ejemplo: descuento $\gamma = 0.5$

- $V([1, 2, 3]) = 1 + 0.5 \times 2 + 0.25 \times 3$
- $V([1, 2, 3]) < V([3, 2, 1])$

Política y retorno

Una **política** $\pi : S \rightarrow A$ le dice al agente qué acción tomar en cada estado: $a = \pi(s)$.

Al ejecutar π desde un estado inicial s_0 , el agente genera una trayectoria:

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots$$

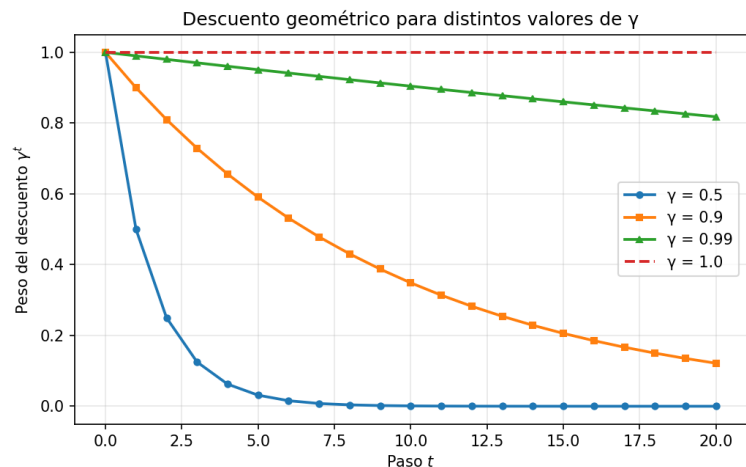
donde cada s_{t+1} se muestrea según $P(\cdot \mid s_t, a_t)$ — es **aleatoria**, no determinista.

El **retorno** de esa trayectoria es la suma de recompensas, **descontada**:

$$G_0 = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots = \sum_{t=0}^{\infty} \gamma^t R(s_t)$$

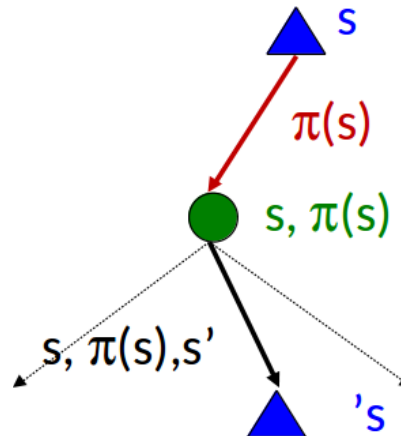
¿Por qué descontar?

- **Convergencia matemática:** sin $\gamma < 1$, esta suma infinita puede no converger si las recompensas no se anulan — con $\gamma < 1$ es una serie geométrica, siempre acotada.
- **Preferencia temporal:** un premio hoy vale más que el mismo premio mañana — la misma lógica que una tasa de interés.
- **Incertidumbre acumulada:** cada paso hacia el futuro es menos seguro que se vaya a realizar tal cual lo proyectamos.



En el Gridworld que vamos a usar hoy, $\gamma = 1$ — la recompensa de vivir $R(s) = -0.04$ en cada paso ya empuja al agente a terminar rápido, así que no necesitamos descuento adicional para garantizar convergencia.

La función de utilidad de una política



Definimos $V^\pi(s)$ como el retorno **esperado** empezando en s y siguiendo π de ahí en adelante:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t) \mid s_0 = s, \pi \right]$$

Necesitamos el valor esperado porque P es estocástica: la misma política, desde el mismo estado, puede generar trayectorias distintas cada vez. $V^\pi(s)$ promedia sobre todas ellas, ponderadas por su probabilidad.

Teorema

Si asumimos preferencias **estacionarias** sobre secuencias de recompensa, es decir, tales que

$$[r_0, r_1, r_2, \dots] \succ [r_0, r'_1, r'_2, \dots] \Rightarrow [r_1, r_2, \dots] \succ [r'_1, r'_2, \dots],$$

entonces solo hay dos formas coherentes de asignar utilidades a las secuencias:

Utilidad aditiva:

$$V([r_0, r_1, r_2, \dots]) = r_0 + r_1 + r_2 + \dots$$

Utilidad descontada: ($0 < \gamma < 1$)

$$V([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$$

Secuencias infinitas

¿Qué pasa si el agente vive para siempre? ¿Obtenemos recompensas infinitas? Comparar secuencias de recompensa con utilidad $+\infty$ es problemático.

Soluciones:

- **Horizonte finito:** (similar a búsqueda con profundidad limitada)
 - Terminar los episodios después de un número fijo de pasos T .
 - Produce políticas no estacionarias (π depende del tiempo restante).
- **Descuento** (con $0 < \gamma < 1$ y recompensas acotadas por $\pm R_{\max}$):

$$V([r_0, r_1, \dots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq \frac{R_{\max}}{1 - \gamma}$$

Un γ más pequeño resulta en un horizonte más corto.

- **Estado absorbente:** garantizar que, para toda política, un estado terminal se alcanza eventualmente.

La observación clave: estacionariedad

El proceso que empieza en s_0 y sigue π **se ve exactamente igual** al proceso que empieza en s_1 y sigue π — misma dinámica, misma política, el mundo no "recuerda" que ya dimos un paso.

Esto significa que $V^\pi(s_1)$, evaluado sobre la cola de la trayectoria a partir de s_1 , es una **copia idéntica** de la definición de V^π — solo que arrancando un paso más tarde. Vamos a usar esta auto-similitud para evitar sumar infinitos términos.

Derivación de la ecuación de Bellman (paso a paso)

Partimos de la definición y separamos el primer término de la cola:

$$V^\pi(s) = \mathbb{E}[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots \mid s_0 = s]$$

Paso 1 — $R(s_0)$ no es aleatorio dado $s_0 = s$: lo sacamos del valor esperado.

$$V^\pi(s) = R(s) + \mathbb{E}[\gamma R(s_1) + \gamma^2 R(s_2) + \cdots \mid s_0 = s]$$

Paso 2 — factorizamos el γ común:

$$V^\pi(s) = R(s) + \gamma \mathbb{E}[R(s_1) + \gamma R(s_2) + \cdots \mid s_0 = s]$$

Derivación de la ecuación de Bellman (paso a paso) — cont.

Paso 3 — por estacionariedad, lo que queda dentro del $\mathbb{E}[\cdot]$ es *exactamente* $V^\pi(s_1)$ evaluado en el estado aleatorio s_1 :

$$V^\pi(s) = R(s) + \gamma \mathbb{E}_{s_1 \sim P(\cdot | s, \pi(s))} [V^\pi(s_1)]$$

Paso 4 — como s_1 es una variable aleatoria discreta con distribución $P(\cdot | s, \pi(s))$, ese valor esperado se escribe explícitamente como una suma ponderada por probabilidad:

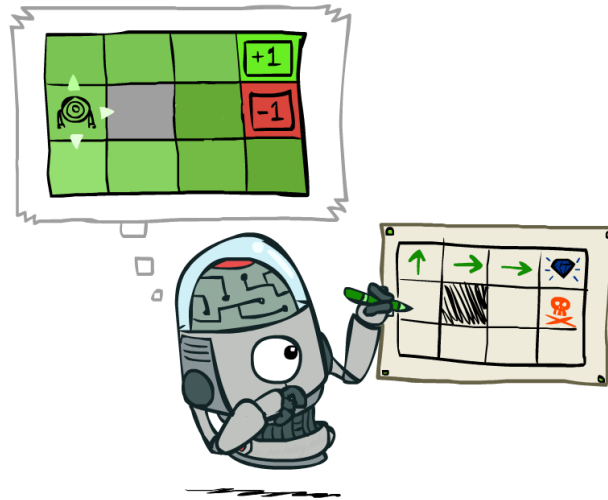
$$V^\pi(s) = R(s) + \gamma \sum_{s' \in S} P(s' | s, \pi(s)) V^\pi(s')$$

Esta es la **ecuación de Bellman**. No es una aproximación — es la definición original, reescrita para no necesitar sumar infinitos términos.

Valores de los estados

La utilidad, o valor, $V(s)$ de un estado se define simplemente como $V^{\pi^*}(s)$.

- Es decir, la recompensa (descontada) esperada si el agente ejecuta una política óptima empezando en s .
- Noten que $R(s)$ y $V(s)$ son cantidades bastante distintas:
 - $R(s)$ es la recompensa de **corto plazo** por haber llegado a s .
 - $V(s)$ es la recompensa total de **largo plazo** desde s en adelante.



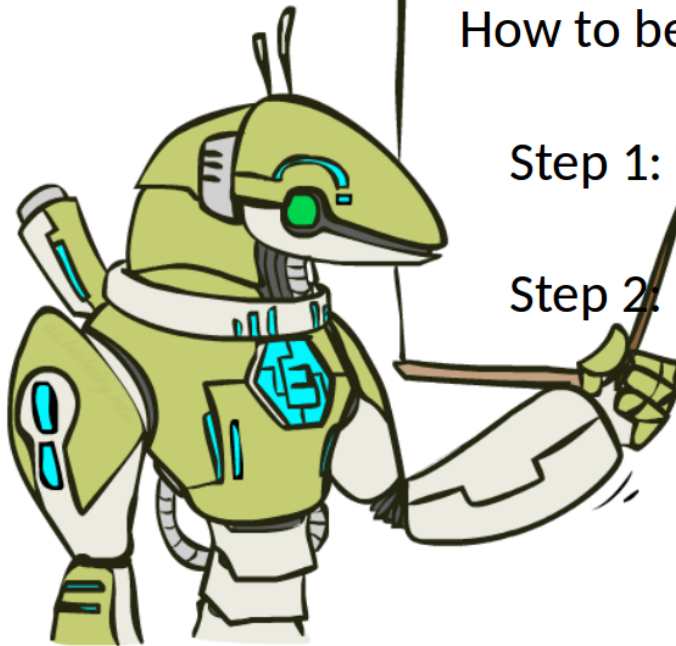
Extracción de política

Usando el principio de máxima utilidad esperada, la acción óptima maximiza la utilidad esperada del estado siguiente. Es decir,

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s' | s, a) V(s').$$

Por lo tanto, podemos extraer la política óptima siempre que podamos estimar las utilidades de los estados.

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s' | s, a) V(s')$$



How to be optimal:

Step 1: Take correct first action

Step 2: Keep being optimal

La política óptima

Ya no fijamos una política — preguntamos: ¿cuál es el mejor retorno posible, sobre **todas** las políticas?

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

Adaptamos la ecuación de Bellman reemplazando "seguir la acción fija $\pi(s)$ " por "elegir la mejor acción posible en cada estado":

$$V^*(s) = R(s) + \max_{a \in A} \gamma \sum_{s' \in S} P(s' \mid s, a) V^*(s')$$

Esta es la **ecuación de Bellman de optimalidad**. La diferencia con la anterior es exactamente el $\max_a$ en vez de una acción fija.

La política óptima se extrae tomando el argmax en lugar del max:

$$\pi^*(s) = \arg \max_{a \in A} \sum_{s' \in S} P(s' \mid s, a) V^*(s')$$

3	0.812	0.868	0.918	+ 1
2	0.762		0.660	-1
1	0.705	0.655	0.611	0.388
	1	2	3	4

Utilidades de los estados en el Gridworld, calculadas con $\gamma = 1$ y $R(s) = -0.04$ para los estados no terminales.

Ejemplo

$$V(1, 1) = -0.04 + \gamma \max[0.8V(1, 2) + 0.1V(2, 1) + 0.1V(1, 1), \\ 0.9V(1, 1) + 0.1V(1, 2), \\ 0.9V(1, 1) + 0.1V(2, 1), \\ 0.8V(2, 1) + 0.1V(1, 2) + 0.1V(1, 1)]$$

Value Iteration

Barrido iterativo de actualización de valores

Value Iteration

Aplicamos la ecuación de Bellman de optimalidad como una **regla de actualización**, repetidamente, sobre todos los estados, hasta que V deje de cambiar:

```
function VALUE-ITERATION(mdp,  $\theta$ ) returns  $V^*$ 
  inicializar  $V(s) := 0$  para todo  $s$ 
  repeat
     $\Delta := 0$ 
    for cada estado  $s$ :
       $V'(s) := R(s) + \max_a \gamma \sum_{s'} P(s' \mid s, a) V(s')$ 
       $\Delta := \max(\Delta, |V'(s) - V(s)|)$ 
     $V := V'$ 
  until  $\Delta < \theta$ 
  return  $V$ 
```

Criterio de parada cuantitativo: paramos cuando el mayor cambio absoluto entre dos barridos (Δ) cae por debajo de un umbral θ — no "cuando se vea estable".

Convergencia

Sean V_i y V_{i+1} aproximaciones sucesivas a la utilidad verdadera V .

Teorema. Para cualesquiera dos aproximaciones V_i y V'_i ,

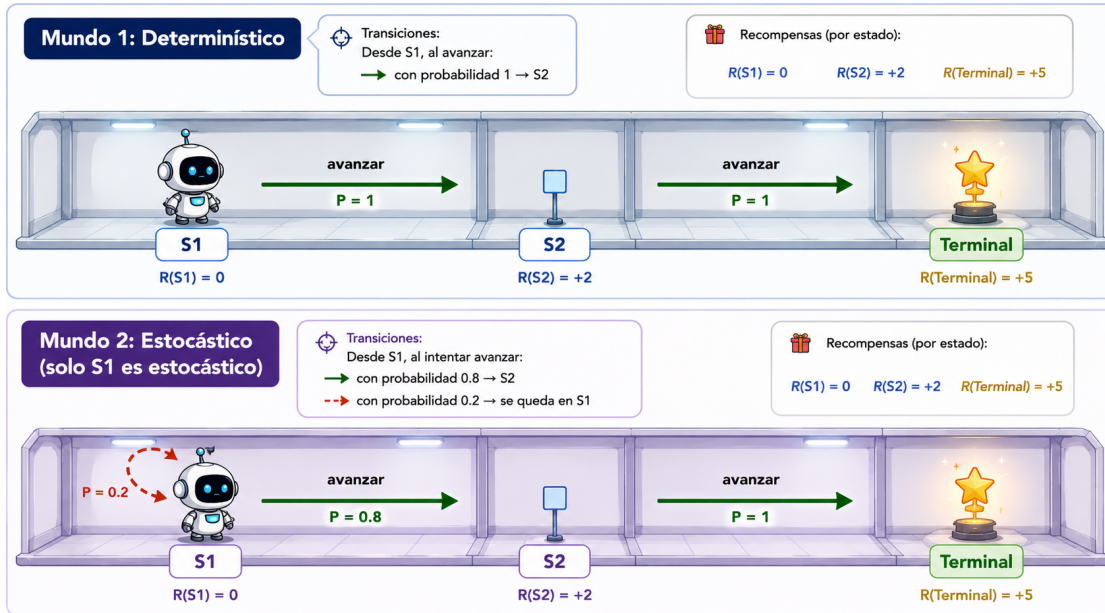
$$\|V_{i+1} - V'_{i+1}\|_{\infty} \leq \gamma \|V_i - V'_i\|_{\infty}.$$

- Es decir, la actualización de Bellman es una contracción por un factor γ en el espacio de vectores de utilidad.
- Por lo tanto, cualesquiera dos aproximaciones deben acercarse entre sí, y en particular cualquier aproximación debe acercarse a la V verdadera.

$\Rightarrow$ Value iteration siempre converge a una única solución de las ecuaciones de Bellman siempre que $\gamma < 1$.

Con $\gamma = 1$ (nuestro caso hoy) la contracción estricta no aplica formalmente, pero la recompensa de vivir negativa garantiza convergencia en la práctica – ver AIMA §17.2 para la discusión completa.

Ejemplo trabajado a mano: franja 1×3



Live-coding: Value Iteration en el Gridworld 4×3

Gridworld 4×3 — layout (AIMA / CS188)

$R(s) = -0.04$	$R(s) = -0.04$	$R(s) = -0.04$	+1
$R(s) = -0.04$	PARED	$R(s) = -0.04$	-1
$R(s) = -0.04$	$R(s) = -0.04$	$R(s) = -0.04$	$R(s) = -0.04$

Ruido: 0.8 acción intentada, 0.1 cada perpendicular | $\gamma = 1$

Actividad — calculen a mano, en parejas

Este es $V^{(2)}$, la tabla de utilidades del Gridworld **después de la iteración 2** de Value Iteration (ya la corrimos en vivo):

-0.0800		-0.0800		+0.7520		+1.0000
-0.0800		WALL		-0.0800		-1.0000
-0.0800		-0.0800		-0.0800		-0.0800

Calculen $V^{(3)}(s)$ para $s = (2, 1)$ — la celda justo debajo del +0.7520, junto al terminal -1. Recuerden: ruido 0.8/0.1/0.1, $R(s) = -0.04$.

3-4 minutos. Después comparamos números.

Solución

$$Q(s, \text{UP}) = 0.8 \cdot V(2, 2) + 0.1 \cdot V(2, 1) + 0.1 \cdot V(3, 1) = 0.8(0.752) + 0.1(-0.08) + 0.1(-$$

$$Q(s, \text{DOWN}) = 0.8 \cdot V(2, 0) + 0.1 \cdot V(2, 1) + 0.1 \cdot V(3, 1) = -0.1720$$

$$Q(s, \text{LEFT}) = 0.8 \cdot V(2, 1) + 0.1 \cdot V(2, 2) + 0.1 \cdot V(2, 0) = 0.0032$$

$$Q(s, \text{RIGHT}) = 0.8 \cdot V(3, 1) + 0.1 \cdot V(2, 2) + 0.1 \cdot V(2, 0) = -0.7328$$

$$V^{(3)}(2, 1) = R(s) + \max_a Q(s, a) = -0.04 + 0.4936 = \mathbf{0.4536}$$

UP gana con claridad — aunque parezca ir "lejos" del terminal +1, es la única acción que no arriesga 10% de probabilidad de caer en el -1.

Policy Iteration

Ciclo evaluar-mejorar de policy iteration

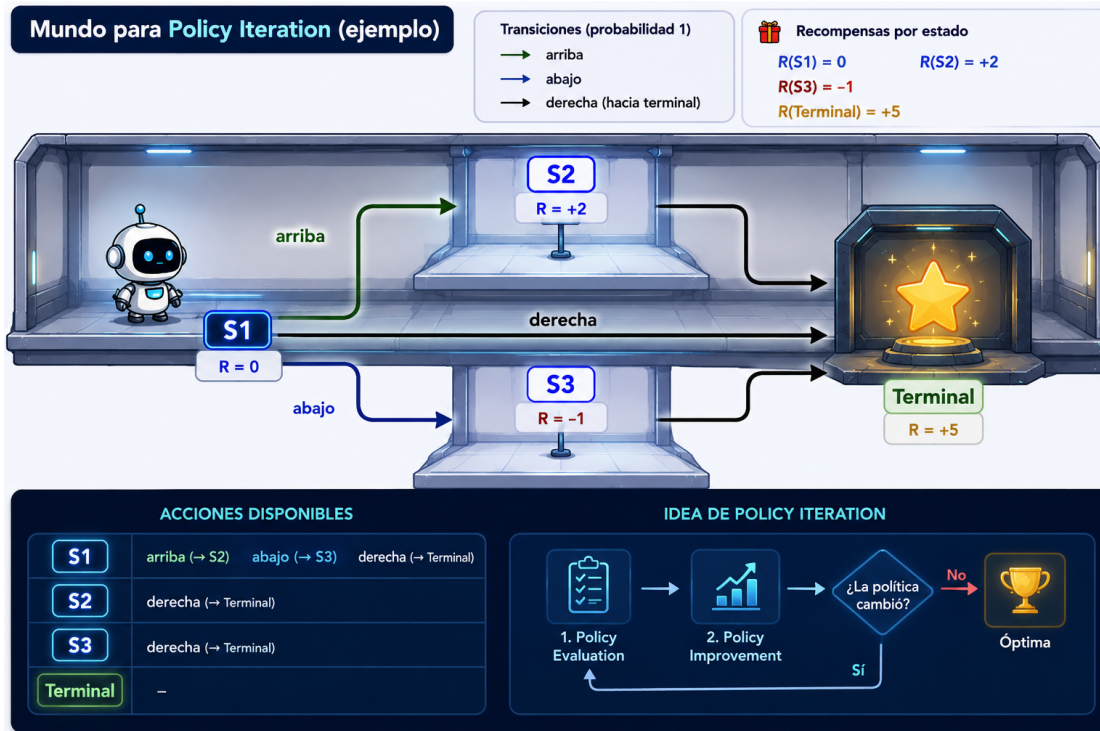
Policy Iteration

En vez de iterar la ecuación de Bellman de *optimalidad* (con el `max` adentro), alternamos entre dos pasos:

```
function POLICY-ITERATION(mdp) returns  $\pi^*$ 
  inicializar  $\pi$  arbitrariamente
  repeat
     $V := \text{EVALUAR-POLITICA}(\pi, \text{mdp})$       # resolver el sistema lineal
     $\pi' := \text{MEJORAR-POLITICA}(V, \text{mdp})$       # greedy respecto a V
    si  $\pi' == \pi$ : return  $\pi$ 
   $\pi := \pi'$ 
```

- **Evaluación de política:** con π fija, la ecuación de Bellman ya no tiene `max` — es un sistema lineal de $|S|$ ecuaciones en $|S|$ incógnitas, resoluble **exactamente**.
- **Mejora de política:** igual que extraer π^* de V^{π^*} , pero con el V^{π} recién calculado.

Ejemplo Policy Iteration

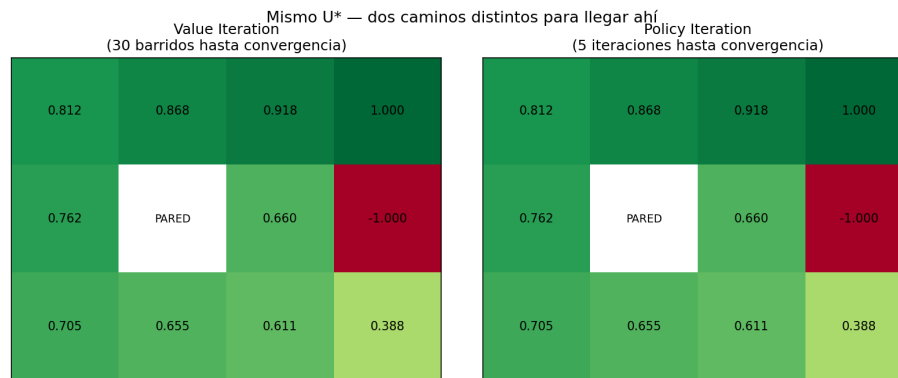


Un dato que vale la pena remarcar

A diferencia de Value Iteration, Policy Iteration con un solver lineal exacto **sí** alcanza la política óptima exacta en un número **finito** de pasos — no solo converge asintóticamente.

- El espacio de políticas es finito y discreto: una vez que π deja de cambiar entre iteraciones, ya es π^* , punto — no hay "casi óptimo" en Policy Iteration.
- Value Iteration, en cambio, siempre tiene un error residual no nulo en V hasta el límite — nunca "termina", solo se acerca lo suficiente según θ .

Live-coding: Policy Iteration en el mismo Gridworld



Value Iteration vs. Policy Iteration

	Value Iteration	Policy Iteration
Qué actualiza	$V(s)$ directamente	Alterna evaluar V^π y mejorar π
Costo por iteración	Barrido sobre estados, más barato	Resolver sistema lineal $ S \times S $, más caro
Convergencia	Asintótica (nunca exacta en tiempo finito)	Exacta en número finito de iteraciones
En nuestro Gridworld (11 estados)	30 barridos	5 iteraciones
MDPs pequeños	Funciona bien	Suele ser el más rápido
MDPs grandes ($ S $ enorme)	Preferido en la práctica	El sistema lineal se vuelve costoso de resolver repetidamente

No hay consenso universal sobre cuál es "mejor" – depende del tamaño de $|S|$. En la práctica, Value Iteration se usa más porque escala mejor cuando el espacio de estados crece.

Cierre: ¿y si no conocemos el modelo?

Todo lo que hicimos hoy —Value Iteration, Policy Iteration— asumió que **conocíamos** $P(s' \mid s, a)$ y $R(s)$ de antemano. El agente nunca interactuó con el mundo: resolvimos el MDP con álgebra pura, dado un modelo ya construido.

¿Qué hacemos cuando **no conocemos** P ni R — cuando la única forma de aprender algo del mundo es **actuando** en él y observando qué pasa? Esa pregunta es exactamente donde empieza Reinforcement Learning, y es el tema completo de la Clase 6.

Resumen

- Un **MDP** formaliza decisión secuencial bajo incertidumbre: (S, A, P, R, γ) .
- La **ecuación de Bellman** no es una aproximación — es la definición de V^π reescrita usando estacionariedad, para evitar sumar infinitos términos.
- **Value Iteration** itera la ecuación de Bellman de optimalidad directamente; converge por ser una contracción, pero solo asintóticamente.
- **Policy Iteration** alterna evaluación exacta (sistema lineal) y mejora codiciosa; converge en menos iteraciones, cada una más cara.
- Ambos asumen que el modelo (P, R) es **conocido** — la pregunta de qué pasa cuando no lo es abre la Clase 6.

Fin de la Clase 5

Próxima clase: Reinforcement Learning — Q-learning, exploración vs. explotación